

Chapter 2

Instructions: Language of the Computer

2.1 Instruction Set

- The repertoire of instructions of a computer
- Different computers have different instruction sets
 - But with many aspects in common
- Early computers had very simple instruction sets
 - Simplified implementation
- Many modern computers also have simple instruction sets

2.2 The RISC-V Instruction Set

- Used as the example throughout the book
- Developed at UC Berkeley as open ISA
- Now managed by the RISC-V Foundation (riscv.org)
- Typical of many modern ISAs
 - See RISC-V Reference Data tear-out card
- Similar ISAs have a large share of embedded core market
 - Applications in consumer electronics, network/storage equipment, cameras, printers, ...

2.2 Arithmetic Operations

- Add and subtract, three operands
 - Two sources and one destination

add a, b, c // a gets b + c
- All arithmetic operations have this form
- *Design Principle 1*: Simplicity favours regularity
 - Regularity makes implementation simpler
 - Simplicity enables higher performance at lower cost

2.2 Arithmetic Example

- C code:

$f = (g + h) - (i + j);$

- Compiled RISC-V code:

```
add t0, g, h    // temp t0 = g + h
add t1, i, j    // temp t1 = i + j
add f, t0, t1   // f = t0 - t1
```

2.3 Register Operands

- Arithmetic instructions use register operands
- RISC-V has a 32×64 -bit register file
 - Use for frequently accessed data
 - 64-bit data is called a “doubleword”
 - 32×64 -bit general purpose registers x0 to x30
 - 32-bit data is called a “word”
- *Design Principle 2: Smaller is faster*
 - c.f. main memory: millions of locations

2.3 RISC-V Registers

- x0: the constant value 0
- x1: return address
- x2: stack pointer
- x3: global pointer
- x4: thread pointer
- x5 – x7, x28 – x31: temporaries
- x8: frame pointer
- x9, x18 – x27: saved registers
- x10 – x11: function arguments/results
- x12 – x17: function arguments

2.3 Reg Operand Example

- C code:

`f = (g + h) - (i + j);`

- `f, ..., j` in `x19, x20, ..., x23`

- Compiled RISC-V code:

`add x5, x20, x21`

`add x6, x22, x23`

`sub x19, x5, x6`

2.3 Memory Operands

- Main memory used for composite data
 - Arrays, structures, dynamic data
- To apply arithmetic operations
 - Load values from memory into registers
 - Store result from register to memory
- Memory is byte addressed
 - Each address identifies an 8-bit byte
- RISC-V is Little Endian
 - Least-significant byte at least address of a word
 - *c.f.* Big Endian: most-significant byte at least address
- RISC-V does not require words to be aligned in memory
 - Unlike some other ISAs

2.3 Memory Operand Example

- C code:

`A[12] = h + A[8];`

- `h` in `x21`, base address of `A` in `x22`

- Compiled RISC-V code:

- Index 8 requires offset of 64

- 8 bytes per doubleword

`ld` `x9, 64(x22)`

`add` `x9, x21, x9`

`sd` `x9, 96(x22)`

2.3 Registers vs. Memory

- Registers are faster to access than memory
- Operating on memory data requires loads and stores
 - More instructions to be executed
- Compiler must use registers for variables as much as possible
 - Only spill to memory for less frequently used variables
 - Register optimization is important!

2.3 Immediate Operands

- Constant data specified in an instruction
`addi x22, x22, 4`
- Make the common case fast
 - Small constants are common
 - Immediate operand avoids a load instruction